

# CodeKids

## Volumen IV: Seguridad, Inteligencia Artificial y Despliegue

**Proyecto:** Plataforma Educativa de Programación para Niños

**Equipo:** Alan / Aquino / Francisco / Alonso

**Periodo:** Enero - Agosto 2026

# 1. Modelo de Seguridad y Privacidad

---

Dada la naturaleza infantil del público objetivo, CodeKids implementa múltiples capas de protección.

## 1.1 Privacidad desde el Diseño (Privacy by Design)

**Sin Registro Público:** La exclusión de un formulario de registro abierto elimina el 99% del riesgo de bots, spam y creación de perfiles falsos por parte de depredadores digitales. El ecosistema es completamente hermético.

## 1.2 Reglas de Firestore (firestore.rules)

Las reglas de seguridad se aplican del lado del servidor de Firebase, impidiendo cualquier inyección de código o manipulación desde la consola del navegador del cliente.

- **Aislamiento de Perfil:** Un estudiante (UID\_A) solo puede leer y modificar su propio documento `/users/UID_A`.
- **Protección de Gamificación:** Variables sensibles como `currentXP` o `level` no pueden ser escritas por el cliente. Solo las funciones seguras del backend pueden incrementarlas tras validar el éxito de una tarea.
- **Seguridad en Comunicaciones:** Los estudiantes no pueden crear chats arbitrarios. Las reglas validan que solo puedan escribir en el canal donde el `teacherId` corresponda al maestro de su grupo.

# 2. Inteligencia Artificial: Asistente "Cody"

---

Cody es un asistente educativo virtual integrado, diseñado para resolver bloqueos cognitivos de los estudiantes durante la programación.

## 2.1 Arquitectura del Motor (Ollama + Proxy)

El sistema emplea un modelo de lenguaje de IA ejecutado localmente utilizando el motor **Ollama**. Para proteger este motor y mantener la integridad del sistema:

1. El estudiante escribe una pregunta en la interfaz de Cody.
2. La solicitud NO va directa a Ollama, sino que es capturada por una **Firestore Function** (Backend serverless).
3. La función actúa como un proxy seguro, inyectando un "System Prompt" invisible que obliga al modelo a: responder en español, usar lenguaje amigable, aplicar metáforas infantiles y negarse a responder sobre temas ajenos a la programación.
4. **Limitación de Tráfico (Rate Limiting):** La función restringe las peticiones a un máximo de **5 solicitudes por hora por dirección IP** para evitar abusos o saturación del servidor local de inferencia.

## 3. Entorno de Desarrollo y Despliegue

---

### 3.1 Emulación Local de Firebase

El equipo de desarrollo emplea el **Firebase Emulator Suite** para evitar trabajar sobre la base de datos de producción y reducir costos durante el ciclo de pruebas. Las herramientas se ejecutan de manera aislada en los siguientes puertos locales (localhost o 127.0.0.1):

- **Auth:** Puerto 9099 - Emulación de identidades y tokens.
- **Firestore:** Puerto 8082 - Base de datos local transaccional.
- **Functions:** Puerto 5001 - Ejecución local de endpoints Node.js.
- **Hosting:** Puerto 5002 - Servidor web estático local.
- **UI Dashboard:** Puerto 4001 - Panel visual para monitorear los emuladores.

### 3.2 Pipelines de Despliegue a Producción

La estrategia de puesta en producción cuenta con redundancia en dos nubes distintas:

#### 1. Infraestructura Principal (Google Firebase):

Mediante el CLI oficial, la ejecución del comando `firebase deploy` sube los estáticos al Hosting CDN de Google, actualiza las Cloud Functions y aplica las reglas de seguridad actualizadas de Firestore/Storage.

#### 2. Infraestructura de Contingencia (Render.com):

En caso de fallo en el ecosistema de Firebase Hosting, el proyecto está configurado a través del archivo declarativo `render.yaml`. Esto conecta el repositorio a Render, habilitando un despliegue alterno automático (CI/CD) de la interfaz de usuario.